Advanced JavaScript for Web Programming: Complete Workflow, Event Listeners, & Exam Solutions

Sayem Shahad, Lecturer, UIU

Objective

This guide covers the exact workflow required to solve UIU JavaScript exam questions. We transition from basic console scripts to fully integrated HTML/JS applications using modern Web Programming standards.

1 Lecture: The Web Programming Workflow

Whenever you face a JavaScript DOM manipulation question in your exam, you should always follow this 4-step framework.

1.1 Step 1: HTML Structure & Script Linking

Never mix your HTML structure with your JavaScript logic. Create a clean user interface using `<input>`, `<button>`, and `<div>` or `<label>` tags for the output. **Crucial Rule:** Always link your `script.js` file at the very bottom of the `<body>`. This ensures the HTML elements exist before JavaScript tries to select them.

```
<body>
    <input type="text" id="myInput">
    <button id="myBtn">Submit</button>
    <div id="output"></div>

    <!-- Link JS right before the closing body tag -->
    <script src="script.js"></script>
</body>
```

1.2 Step 2: Global State Management (Memory)

If the exam asks you to "keep track" of something (like the number of attempts or a running total of calories), you MUST declare these variables **outside** of any functions. If you declare them inside the function, they will reset to zero every time the user clicks the button!

```
// GLOBAL SCOPE: Remembers values across multiple button clicks
let totalAttempts = 0;
let runningScore = 0;
```

1.3 Step 3: Event Listeners & Data Extraction

Instead of adding `onclick=""` inside your HTML, use `addEventListener` in your JS file. Inside this listener, extract the user's input using `.value`. Remember to convert text to numbers if you plan to do math!

```
document.getElementById("myBtn").addEventListener("click", function() {
    // Grab the input element and read its value
    let inputString = document.getElementById("myInput").value;

    // Convert to a number (Crucial for math/comparisons)
    let userNumber = parseInt(inputString);

    // Validate input to prevent errors
    if (isNaN(userNumber)) return;

    totalAttempts++; // Update global state
});
```

1.4 Step 4: Specialized Logic & Updating the UI

Once you process the logic, you must push the result back to the HTML screen.

- **Updating Text:** Use `.innerHTML` (if you want to include `<br>` or spans) or `.innerText`.
- **Disabling Elements:** Use `.disabled = true` to lock inputs or buttons (often required in games or password checkers after a certain number of tries).
- **Regex (.test):** Use `/[A-Z]/.test(string)` to check if a string contains uppercase letters (or lowercase, numbers, etc.).
- **Math.random:** Use `Math.floor(Math.random() * (Max - Min + 1)) + Min` to generate whole random numbers.

2 Complete Exam Solutions

The following solutions apply the 4-step workflow to your specific past exam papers.

2.1 Exam 1: Calorie Tracker (Summer 2025)

Focus: Global state tracking and complex `if/else` logic.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Calorie Tracker</title>
</head>
<body>
    <h2>Daily Calorie Tracker</h2>
    <input type="number" id="calInput" placeholder="Calories per meal">
    <button id="addBtn">Add Calories</button>

    <div id="feedbackLabel" style="margin-top: 15px;">
        Total: 0 | Feedback: Waiting for input...
    </div>

    <script src="script.js"></script>
</body>
</html>
```

index.html

```javascript
// 1. Global State Management
const GOAL = 2000;
let totalCalories = 0;
let entryCount = 0;

// 2. Attach Event Listener to Button
document.getElementById("addBtn").addEventListener("click", function()
    {

    let inputField = document.getElementById("calInput");
    let mealCal = parseInt(inputField.value);

    // Validation
    if (isNaN(mealCal) || mealCal <= 0) return;

    // Update state
    totalCalories += mealCal;
    entryCount++;

    // Determine feedback
    let msg = "";
    if (totalCalories <= 800) {
        msg = "You're off to a healthy start!";
    } else if (totalCalories <= 1600) {
        msg = "Good progress, keep it balanced!";
    } else if (totalCalories <= 1999) {
        msg = "Almost at your limit!";
    } else {
        msg = "Goal reached! Stay mindful!";
```

```
    }

    // Override feedback for frequent snacking
    if (entryCount > 10 && totalCalories < GOAL) {
        msg = "Be cautious of frequent snacking!";
    }

    // Render to DOM
    document.getElementById("feedbackLabel").innerHTML =
        `Total Calories: ${totalCalories} / ${GOAL} <br> Feedback: ${
           msg}`;

    // Clear input field for next entry
    inputField.value = "";
});
```

script.js

2.2 Exam 2: Number Guessing Game (Fall 2024)

Focus: Math Object, element disabling, and game state loops.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Number Guessing Game</title>
</head>
<body>
    <h2>Guess the Secret Number</h2>
    <input type="text" id="guessInput" placeholder="Enter guess (500
       -5000)">
    <button id="submitBtn">Submit</button>

    <div id="resultLabel" style="margin-top: 15px; font-weight: bold;">
        Awaiting your first guess...
    </div>

    <script src="script.js"></script>
</body>
</html>
```

index.html

```javascript
// 1. Generate target & track global attempts
let secretNumber = Math.floor(Math.random() * (5000 - 500 + 1)) + 500;
let guesses = 0;

// 2. Attach Event Listener
document.getElementById("submitBtn").addEventListener("click", function
    () {

    let inputField = document.getElementById("guessInput");
    let resultLabel = document.getElementById("resultLabel");
    let userGuess = parseInt(inputField.value);

    if (isNaN(userGuess)) return;

    guesses++; // Increase attempt count

    // Check if user runs out of guesses BEFORE evaluating correctness
    if (guesses >= 5 && userGuess !== secretNumber) {
        resultLabel.innerText = "Out of guesses!";
        inputField.disabled = true; // Lock input
        this.disabled = true;       // Lock button
        return;
    }

    // Evaluate Game Logic
    if (userGuess > secretNumber) {
        resultLabel.innerText = "Too high!";
    } else if (userGuess < secretNumber) {
        resultLabel.innerText = "Too low!";
    } else if (userGuess === secretNumber) {
        resultLabel.innerText = "Correct!";
        inputField.disabled = true;
        this.disabled = true;
```

```
    }
});
```

script.js

2.3 Exam 3: Password Strength Checker (Spring 2025)

Focus: String manipulation, Regular Expressions (Regex), and complex scoring.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Password Strength Checker</title>
</head>
<body>
    <h2>Password Strength Evaluator</h2>
    <input type="password" id="passInput" placeholder="Enter your
       password">
    <button id="checkBtn">Check Strength</button>

    <div id="strengthLabel" style="margin-top: 15px;">
        Score: 0 | Strength: None
    </div>

    <script src="script.js"></script>
</body>
</html>
```

index.html

```javascript
// Global state tracking
let attempts = 0;

// Event Listener integration
document.getElementById("checkBtn").addEventListener("click", function
    () {

    let pw = document.getElementById("passInput").value;
    let label = document.getElementById("strengthLabel");

    attempts++;
    let score = 0;
    let strengthText = "";

    // Minimum 6 characters rule
    if (pw.length < 6) {
        label.innerText = "Error: Minimum 6 characters required.";
        return;
    }

    // Calculate Base & Regex Scores
    score += Math.floor(pw.length / 2) * 10; // +10 points per 2 chars

    if (/[A-Z]/.test(pw)) score += 15; // Uppercase test
    if (/[a-z]/.test(pw)) score += 15; // Lowercase test
    if (/[0-9]/.test(pw)) score += 20; // Number test
    if (/[!@#$%^&*]/.test(pw)) score += 25; // Special Character test

    // Determine Level Category
    if (score <= 30) { strengthText = "Very Weak"; }
    else if (score <= 50) { strengthText = "Weak"; }
    else if (score <= 70) { strengthText = "Medium"; }
    else if (score <= 90) { strengthText = "Strong"; }
```

```
    else { strengthText = "Very Strong"; }

    // Special Overrides
    if (score >= 100) {
        strengthText = "Perfect Password!";
    }

    // Specific Attempt Condition based on Exam Rule
    if (attempts > 8 && score <= 70) {
        label.innerHTML = `Need practice! Suggestion: Add uppercase,
            numbers, and special characters.`;
        return;
    }

    // Print Final Result
    label.innerHTML = `Score: ${score} | Strength: ${strengthText}`;
});
```

script.js